

DSA Pattern Guide: Recursion, Backtracking, and Dynamic Programming

This guide summarizes the core patterns for Recursion, Backtracking, and Dynamic Programming commonly used in coding interviews and competitive programming. It also includes templates and a quick recognition strategy for identifying patterns during problem solving.

1. Recursion Fundamentals

- Base Case: Stop recursion when condition is met.
- Recursive Case: Reduce problem size and call function again.
- Work may occur before or after recursion.
- Common uses: factorial, power, Fibonacci, tree traversal.

2. Backtracking Core Idea

Backtracking explores all possibilities and reverses choices after exploring a branch. Core structure: choose → recurse → undo.

Template:

```
for(choice in choices):  
    make choice  
    recurse  
    undo choice
```

Major Backtracking Patterns

Pattern	Idea	Complexity	Example Problems
Subsets / Subsequences	Take or not take each element	$O(2^n)$	Subsets (LC 78)
Combinations	Choose next element from remaining #s	$O(2^n)$	Combination Sum (LC 39)
Permutations	Choose unused element each position	$O(n!)$	Permutations (LC 46)
Constraint Backtracking	Generate but prune invalid states early	Varies	Generate Parentheses (LC 22)

3. Dynamic Programming (DP)

- Used when problems have overlapping subproblems and optimal substructure.
- Two approaches: Memoization (top-down recursion + cache) and Tabulation (bottom-up).
- Often derived by converting recursion into a table.

Typical DP Template:

dp[0] = base case

for state in states:

for option in transitions:

dp[state] = best(dp[state], dp[previous_state] + cost)

4. 10-Second Pattern Recognition Trick

- If each element has two choices (take / skip) → Subset pattern.
- If selecting some elements in order without reuse → Combination pattern.
- If arranging elements and order matters → Permutation pattern.
- If recursion repeats same states → Dynamic Programming.

5. Recommended Learning Order

- Recursion Basics (factorial, power, Fibonacci)
- Subsequence / Subset problems
- Combination Sum family
- Permutations
- Constraint backtracking (Parentheses, N-Queens)
- Dynamic Programming introduction
- Classic DP problems (Knapsack, LIS, Coin Change)